



Unidad 2

Colaboración entre objetos y miembros de clase

Prof: Lic. Rosales Pablo (prosales@unpata.edu.ar)
Materia: Programación Orientada a Objetos
Año: 2026



Lo que acaban de hacer



- Cuatro estudiantes creados, y los cuatro con el legajo 10001. Un curso con cupo para 2, y terminó con 4 inscriptos.
- El programa no falló ni avisó nada. Compiló, corrió y mintió.

¿Quién tendría que haber impedido cada una de las dos cosas: el main que crea e inscribe, o las clases que hicieron la promesa?

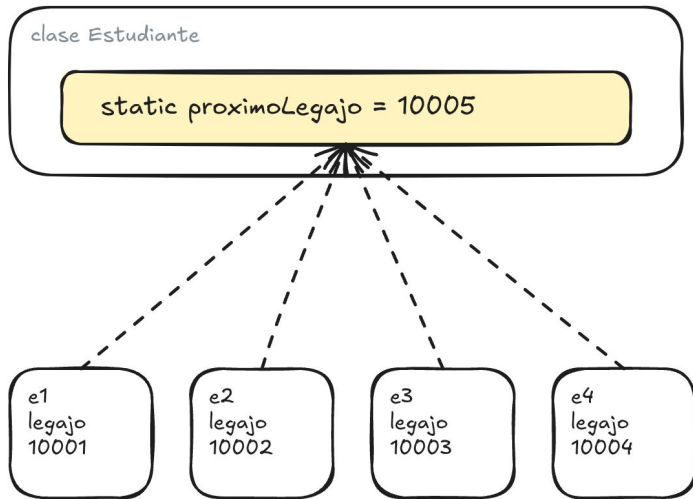
La línea del constructor parecía correcta: `legajo = proximoLegajo++`. El error no estaba en esa línea: estaba en dónde vive el contador.

El legajo y la memoria de clase (static)

Instancia y clase (static)

Declarado como miembro de instancia, cada estudiante nace con su propia copia de `proximoLegajo`. Cuatro objetos, cuatro contadores, todos en 10001.

- `static` lo convierte en un miembro de clase: una única copia, que vive en la clase `Estudiante` y es compartida por todos sus objetos.
- Cada `new Estudiante(...)` toma el valor actual del contador compartido y lo incrementa para el siguiente, se cree donde se cree.



una única copia, compartida por todos los objetos;
el legajo de cada uno sigue siendo de instancia

El legajo de cada estudiante sigue siendo de instancia. Lo único compartido es el próximo a asignar: un dato que no es de ningún estudiante, sino de todos a la vez.

Cómo se accede a un miembro de clase



Por la clase

- `Estudiante.getProximoLegajo()`.
No hace falta ningún objeto: el dato es de la clase.
- Adentro de un método `static` no existe `this`: no hay ningún objeto sobre el que estar parado.

Por un objeto

- `e1.getProximoLegajo()` **compila**, pero es mala práctica.
- El que lee cree que consulta un dato de `e1`, y en realidad consulta un dato de toda la clase.

Ya venían usando miembros de clase: `Math.PI`, `Integer.parseInt(...)` y el propio `main`, que es `static` porque al arrancar el programa todavía no existe ningún objeto.

El curso y el estado de instancia



De cada curso

- `cupoMaximo` y la lista `inscriptos`. El curso de P00 con cupo 2 es independiente de un curso de Física con cupo 50.
- `this.inscriptos` es la lista de ese curso, no la de la facultad.

De la clase Estudiante

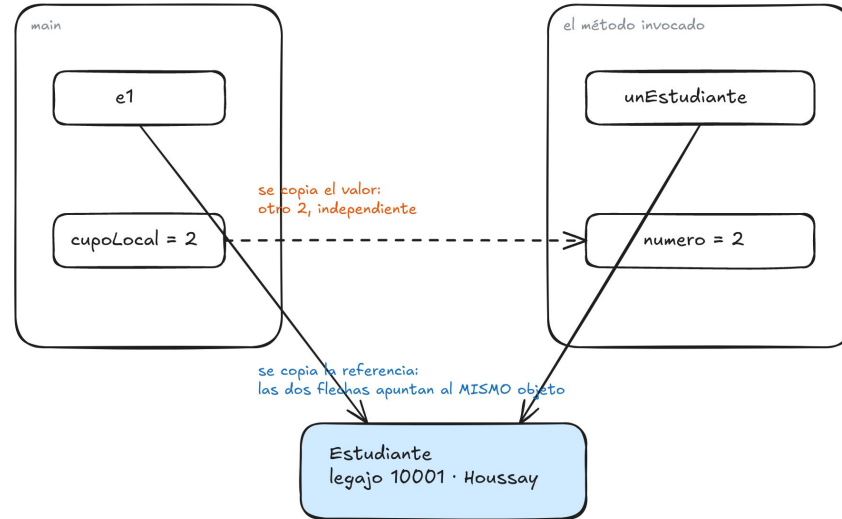
- `proximoLegajo`, el numerador compartido de la facultad entera.
- ¿Qué pasaría si por error `inscriptos` fuera `static`? Todos los cursos compartirían una única lista de `inscriptos`.

La pregunta de diseño de hoy, para cada dato: ¿de quién es? Si es de cada objeto, instancia. Si es de todos a la vez, clase.

La inscripción y el paso por referencia

- Al ejecutar `poo.inscribirEstudiante(e1)` el objeto no viaja: el parámetro recibe una copia de la referencia, la dirección para encontrar a ese único objeto.
- Por eso la lista `inscriptos` guarda al mismo estudiante que la variable `e1` del `main`. No hay dos Houssay: hay uno solo con dos nombres.
- La prueba: `e1.cambiarApellido(...)` no tocó el curso, y el listado del curso mostró el apellido nuevo.

Qué se copia al pasar un parámetro



`contains` también trabaja con referencias: pregunta si la lista ya guarda a ese mismo objeto. Por eso alcanzó para detectar la reinscripción.

Java siempre pasa por valor



Primitivos

- El valor es el dato mismo.
`intentarModificar(cupoLocal)`
recibió una copia del 2.
- Adentro del método valió 4;
afuera siguió valiendo 2.

Objetos

- El valor de la variable es una referencia. Se copia la referencia, no el objeto: dos variables, un solo objeto.
- Reasignar el parámetro (`destino = new Billetera(...)`) no cambia la variable de quien llamó.

La respuesta de final: Java pasa siempre por valor; con objetos, lo que se copia es la referencia. «Los objetos se pasan por referencia» es casi correcto, y por eso es tan común.

Colaborar sin romper el encapsulamiento



Lo que un objeto puede

- Mandar mensajes:
`destino.cargar(monto),`
`unEstudiante.getApellido().`
- Pedir, y que el otro decida con sus propias reglas.

Lo que no puede

- Tocar el estado ajeno: `destino.saldono` compila, es `private`.
- La colaboración no abre ninguna puerta que el encapsulamiento haya cerrado.

Un programa orientado a objetos es una comunidad: objetos que se piden cosas por sus interfaces públicas. Eso son los mensajes.

Instancia o clase: cómo decidir



- **Pertenencia** instancia: cada objeto la suya. static: una sola, de la clase.

Analogía el legajo es de cada estudiante; el nombre de la universidad es de todos.

Acceso objeto.miembro para instancia; NombreClase.miembro para static.

- **this** disponible en métodos de instancia; inexistente en contexto static.

La tabla completa está en la lectura previa. La pregunta «¿de quién es este dato?» se responde antes de escribir la palabra static, no después.

Cuándo usar static (y cuándo no)



- Sí: contadores y numeradores compartidos (`proximoLegajo`), constantes (`Math.PI`), utilidades sin estado (`Integer.parseInt`).
- No: casi todo lo demás. Un atributo static es estado global: cualquiera puede modificarlo, y cuando algo sale mal no se sabe quién fue, porque el dato no tiene dueño.
- La regla práctica: si un dato es de un objeto, va en el objeto. static es la excepción que hay que poder justificar, no una opción más.

En el proyecto integrador va a haber muy pocos static, y cada uno va a tener que poder explicar por qué existe.

Una comunidad de objetos

Cada clase del proyecto integrador va a guardar referencias, recibir objetos por parámetro y decidir qué es de cada objeto y qué es de todos.
